

CSE330 Numerical Methods – Lab Final

Demo / practice question · This is not a real exam question

Codes for importing the necessary libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial import Polynomial
```

Question 01

[8 Marks]

The **forward** and **backward** difference functions are already given to you:

```
def forward_diff(f, x, h):
    return (f(x + h) - f(x)) / h

def backward_diff(f, x, h):
    return (f(x) - f(x - h)) / h
```

A. [2 Marks] Using `forward_diff` and `backward_diff`, write a function `central_diff(f, x, h)` that returns the central difference approximation of $f'(x)$.

B. [3 Marks] Implement these two formulas for $f'(x)$ as functions `formula_A(f, x, h)` and `formula_B(f, x, h)`:

Formula A (three-point forward): $f'(x) \approx [-3f(x) + 4f(x+h) - f(x+2h)] / (2h)$

Formula B (five-point central): $f'(x) \approx [f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)] / (12h)$

C. [3 Marks] Let $f(x) = \sin(x)$, $x = \pi/4$ and $h = 0.2$. The actual derivative is $\cos(x)$. Find $f'(x)$ using the central difference, Formula A and Formula B. Find the absolute error of Formula A and Formula B, and print which formula is better.

Hint: In numpy, use `np.sin(x)`, `np.cos(x)` and `np.pi`. Pass the function itself, for example `central_diff(np.sin, x, h)`. Absolute value: `abs(...)`.

YOUR CODE SHOULD GIVE OUTPUT

```
Central difference : 0.7024021550949058
Formula A error    : 0.007891838885678792
Formula B error    : 3.753319739907557e-05
Better formula     : B
```

Question 02

[6 Marks]

A polynomial $p(x) = c_0 + c_1x + c_2x^2 + \dots$ can be found with a **Vandermonde-type matrix** A of size $n \times n$, where:

$$A_{ij} = x_i^{j+1} \quad \text{for } i, j = 0, 1, \dots, n-1$$

Then we solve $A \cdot c = y$ to get the coefficients c .

A. [2 Marks] Write a function `make_vandermonde(x)` that returns the matrix A for a given array x .

B. [2 Marks] Find the coefficients using numpy and build the polynomial using the **Polynomial class**. Print the coefficients and the polynomial.

C. [2 Marks] Find $p(4)$. Then plot $p(x)$ with a **green line** for $-0.5 \leq x \leq 4$.

Hint: Find the coefficients with the inverse: $c = A^{-1} \cdot y$. First find the inverse with `np.linalg.inv(A)`, then multiply using `np.dot(A_inv, y)`. Use `np.linspace(-0.5, 4, 100)` to make the x -values for the plot.

SAMPLE INPUT

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 4, 60, 1296])
```

SAMPLE OUTPUT

Matrix:

```
[[ 1.  0.  0.  0.]
 [ 1.  1.  1.  1.]
 [ 4.  8. 16. 32.]
 [27. 81.243.729.]]
```

Coefficients:

```
[ 3.00000000e+00 -1.0658141e-14
 -1.00000000e+00  2.00000000e+00]
p(4) = 114.99999999999999
```

Question 03

[6 Marks]

Let the nodes be $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. The **Lagrange interpolation polynomial** is:

$$P(x) = \sum y_k \cdot l_k(x) \quad \text{for } k = 0, 1, \dots, n$$

where $l_k(x)$ is the **Lagrange basis function**:

$$l_k(x) = \prod (x - x_j) / (x_k - x_j) \quad \text{for } j = 0, 1, \dots, n \text{ and } j \neq k$$

Remember: $\sum$ means **add** all the terms, and $\prod$ means **multiply** all the terms.

A. [3 Marks] Complete the function `l_k` for the Lagrange basis. Multiply the fraction $(x - x_j) / (x_k - x_j)$ for every j , but skip $j = k$.

```
def l_k(data_X, k, x):  
    result = 1.0  
    for j in range(len(data_X)):  
        # Your code here  
  
    return result
```

B. [3 Marks] Complete the function `get_poly_lagrange` using `l_k`. For every value in `x_arr`, add $y_k \cdot l_k(x)$ for all k .

```
def get_poly_lagrange(data_X, data_Y, x_arr):  
    answers = []  
    for x_value in x_arr:  
        total = 0.0  
        # Your code here  
  
    answers.append(total)  
    return np.array(answers)
```

SAMPLE INPUT

```
data_X = np.array([1, 2, 4])  
data_Y = np.array([3, 5, 4])  
x_arr = np.array([0, 3, 5])  
k = 1, x = 3 (for l_k)
```

SAMPLE OUTPUT

```
l_1(3) = 1.0  
  
P(x_arr) =  
[-0.66666667  5.33333333  1.          ]
```

Question 04

[6 Marks]

A sensor follows the function $g(x) = \cos(x) + x$. The nodes are given only as x-values: `hx = np.array([0, 1, 2])`. You must find the y-values and the y'-values yourself.

Hint: $g'(x) = -\sin(x) + 1$. In numpy, use `np.cos(x)` and `np.sin(x)`.

A. [2 Marks] Using numpy, find `hy` (values of g at the nodes) and `hdy` (values of g' at the nodes).

B. [4 Marks] Implement the following formulas for the Hermite interpolation. Then find $H(1.5)$ and compare it with the actual $g(1.5)$.

$$l_k(x) = \prod (x - x_j) / (x_k - x_j) \quad \text{for } j \neq k$$

$$l'_k(x_k) = \sum 1 / (x_k - x_j) \quad \text{for } j \neq k$$

$$h_k(x) = [1 - 2(x - x_k) \cdot l'_k(x_k)] \cdot l_k(x)^2$$

$$\hat{h}_k(x) = (x - x_k) \cdot l_k(x)^2$$

$$H(x) = \sum [y_k \cdot h_k(x) + y'_k \cdot \hat{h}_k(x)] \quad \text{for } k = 0, 1, 2$$

YOUR CODE SHOULD GIVE OUTPUT

```
hy = [1.          1.54030231  1.58385316]
hdy = [1.          0.15852902  0.09070257]
```

```
H(1.5)          = 1.5708271856736837
Actual g(1.5) = 1.5707372016677028
```

Final Exam Tips

Read these before your lab final

- 1 Focus on the “Your code here” part.**
Mainly focus on the “Your code here” part of the lab sheet. This is where the exam questions come from.
- 2 Watch all the lab videos.**
Watch all the lab videos I made. I only focused on the “Your code here” part, nothing more.
- 3 Always try to code something.**
After the exam there is always a viva. So even if you do not understand the question, try to code something.
- 4 If you still cannot do it, write a comment.**
If you really cannot write the code, explain your idea in a comment. Do not leave it empty.
- 5 Click “Run All”.**
Make sure to click “Run All” so that numpy and matplotlib are also run. Otherwise you will get errors.
- 6 Be careful with indentation.**
Python depends on indentation. One wrong space can break your whole code.

Good luck with your exam!

Topic List

Topics covered in the lab course

Lab2: Polynomial Interpolation (Vandermonde)

Lab3: Lagrange Interpolation

Lab4: Newton's Divided Difference

Lab5: Hermite Interpolation

Lab6: Differentiation and Richardson

Lab6: Nonlinear Equations (Bisection and Fixed Point Iteration)

Lab7: Solving Linear System
